

The MongoDB Aggregation Pipeline

[Try MongoDB Atlas Free](#)

Table of Contents

- [What is the Aggregation Pipeline in MongoDB?](#)
- [MongoDB Aggregation Pipeline Example](#)
- [What are the Aggregation Pipeline Operators in MongoDB?](#)
- [What are the Aggregation Stages in MongoDB?](#)
- [How to Optimize Performance in MongoDB Aggregation](#)

When you start doing advanced queries in MongoDB, you realize that the basic `find()` command won't give you the flexibility and robustness that you need. Fear not: The aggregation pipeline, a multi-stage pipeline that transforms the documents into aggregated results, is here to help.

There are three ways to perform aggregation in MongoDB:

- the map-reduce function
- single-purpose aggregation
- the aggregation pipeline

Map-reduce uses custom JavaScript functions to perform the map and reduce operations. This method fails to provide a simple interface, requiring you to implement JavaScript within MongoDB, and thus suffers from performance overhead.

Single-purpose aggregation provides simple access to common aggregation processes like counting the number of documents within a single collection and/or returning unique documents. But their biggest downside is that they lack the flexibility and capabilities of the aggregation pipeline and map-reduce.

The aggregation pipeline is often preferred and the recommended way of doing aggregations in MongoDB. It is designed specifically to improve performance and usability for aggregation. Pipeline operators need not produce one output document for every input document, but can also generate new documents or filter out documents. Moreover, starting from MongoDB version 4.4, it can also define custom aggregation expressions with `$accumulator` and `$function`.

What is the Aggregation Pipeline in MongoDB?

The aggregation pipeline refers to a specific flow of operations that processes, transforms, and returns results. In a pipeline, successive operations are informed by the previous result.

Let's take a typical pipeline:

Input -> `$match` -> `$group` -> `$sort` -> output

In the above example, input refers to one or more documents. `$match`, `$group`, and `$sort` are various stages in a pipeline. The output from the `$match` stage is fed into `$group` and then the output from the `$group` stage into `$sort`. These three stages collectively can be called an aggregation pipeline.

Implementing a pipeline helps us to **break down queries into easier stages**. Each stage uses namesake operators to complete transformation so that we can achieve our goal.

While there is no limit to the number of stages used in the query, it is worth noting that the order of the stages matters and there are optimizations that can help your pipeline perform better. For instance, a `$match` at the beginning of the pipeline greatly improves overall performance.

MongoDB Aggregation Pipeline Example

Let's consider a collection called *students* as a record of people signing up for online courses. This will be our input collection. Students are given an `id` (a unique value), their classes, section, and the course fee. The document based on this data model would look something like this:

```
{
  student_id: "P0001",
  class: 101,
  section: "A",
  course_fee: 12
},
{
  student_id: "P0002",
  class: 102,
  section: "A",
  course_fee: 8
},
{
  student_id: "P0002",
  class: 101,
  section: "A",
  course_fee: 12
},
{
  student_id: "P0004",
  class: 103,
  section: "B",
  course_fee: 19
}
```

Now, if we run the `aggregate()` function on a mongo shell or MongoDB Compass, to calculate the total course fee of all the students in *Section A*, we will have to create a query similar to below:

```
db.students.aggregate([
  { $match: { section: "A" } },
  { $group: { student_id: "student_id", total: { $sum: "$course_fee" } } }
])
```

In the above example, the `$group` stage would use the output of the previous aggregate, which is `$match` as input for the next function.

If we execute this query, we will get the following result.

```
{
  student_id : "P001",
  total: 12
},
{
  student_id : "P002",
  total: 20
}
```

Here, the final results show the total number of students. The computed fields are *studentid* and *the total*. In this query, we have used `$match` to limit the students to *Section A*. Then, we have grouped the students by *studentid* and calculated the sum total of *coursefee*. In this example, we have used an aggregation pipeline to transform the *coursefee*.

If we were to express the above example through a pipeline, it would appear as: `students => $match => $group => desired result`.

Depending on your needs, you can add other aggregate functions at any stage in the pipeline. But remember, placing a match stage at the beginning limits the total number of documents in the pipeline and reduces the processing time.

Query can also take advantage of indexes if you put the match at the very beginning of a pipeline. The sort stage usually performs better at the end because keeping it otherwise often means additional calculations or aggregations that might be performed can affect the sort order, hence rendering the output of the sort stage irrelevant.

Through MongoDB Compass, MongoDB also allows you to graphically create your aggregation pipeline. Visit [Aggregation Pipeline Builder](#) for more information.

What are the Aggregation Pipeline Operators in MongoDB?

MongoDB offers an expansive list of operators that you can use across various aggregation stages. Each of these operators can be used to construct expressions for use in the aggregation pipeline stages.

Operator expressions are similar to functions that take arguments. In general, these expressions take an array of arguments and have the following form:

```
{ <operator> : [ <argument1>, <argument2>, ... ] }
```

In case you only want to use an operator that accepts a single argument, you can omit the array field. It will take the following form:

```
{ < operator> : <argument> }
```

These operators are available to construct expressions for use in the aggregation pipeline stages. (For more information on each of these operators, visit [Aggregation Pipeline Operators](#).)

- Arithmetic Expression Operators** perform mathematical operations on numbers.
- Array Expression Operators** perform operations on arrays.
- Boolean Expression Operators** evaluate their argument expressions as booleans and return a boolean as the result.
- Comparison Expression Operators** return a boolean, except for `$cmp`, which returns a number.
- Conditional Expression Operators** help build conditional statements.
- Custom Aggregation Expression Operators** define custom aggregation functions.
- Data Size Operators** return the size of a data element.
- Date Expression Operators** return date objects or components of a date object.
- Literal Expression Operators** return a value without parsing.
- Object Expression Operators** split/merge documents.
- Set Expression Operators** perform set operation on arrays, treating arrays as sets.
- String Expression Operators** perform well-defined behavior for strings of ASCII characters.
- Text Expression Operators** allow access to per-document metadata related to the aggregation.
- Trigonometry Expression Operators** perform trigonometric operations on numbers.
- Type Expression Operators** perform operations on data type.
- Accumulators** (`$group`) maintain state as the document progresses through the pipeline.
- Accumulators (in Other Stages)** do not maintain their state and can take as input either a single argument or multiple arguments.
- Variable Expression Operators** define variables for use within the scope of a subexpression and return the result of the subexpression.

What are the Aggregation Stages in MongoDB?

Each stage of the aggregation pipeline transforms the document as the documents pass through it. However, once an input document passes through a stage, it doesn't necessarily produce one output document. Some stages may generate more than one document as an output.

MongoDB provides the `db.collection.aggregate()` method in the mongo shell and the `db.aggregate()` command to run the aggregation pipeline.

A stage can appear multiple times in a pipeline, with the exception of `$out`, `$merge`, and `$geoNear` stages. In this article, we will discuss in brief the seven major stages that you will come across frequently when aggregating documents in MongoDB. For a list of all available stages, see [Aggregation Pipeline Stages](#).

- \$project**
 - Reshapes each document in the stream, e.g., by adding new fields or removing existing fields. For each input document, output one document.
- \$match**
 - Filters the document stream to allow only matching documents to pass unmodified into the next pipeline stage. For each input document, the output is either one document (a match) or zero document (no match).
- \$group**
 - Groups input documents by a specified identifier expression and apply the accumulator expression(s), if specified, to each group. `$group` consumes all input documents and outputs one document per each distinct group. The output documents only contain the identifier field (`group_id`) and, if specified, accumulated fields.
- \$sort**: Reorders the document stream by a specified sort key. The documents are unmodified, except for the order of the documents. For each input document, the output will be one document.
- \$skip**
 - Skips the first *n* documents where *n* is the specified skip number and passes the remaining documents unmodified to the pipeline. For each input document, the output is either zero document (for the first *n* documents) or one document (after the first *n* documents).
- \$limit**
 - Passes the first *n* documents unmodified to the pipeline where *n* is the specified limit. For each input document, the output is either one document (for the first *n* documents) or zero document (after the first *n* documents).
- \$unwind**
 - Breaks an array field from the input documents and outputs one document for *each* element. Each output document will have the same field, but the array field is replaced by an element value per document. For each input document, outputs *n* documents where *n* is the number of array elements and can be zero for an empty array.

How to Optimize Performance in MongoDB Aggregation

One of the best features of the MongoDB aggregation pipeline is that it automatically reshapes the query to improve its performance. Having said that, here are a few things to consider for optimized query performance.

- Pipeline stages have a limit of 100 megabytes of RAM. If a stage exceeds this limit, MongoDB will produce an error. To allow for the handling of large datasets, use the `allowDiskUse` option to enable aggregation pipeline stages to write data to temporary files. Keep in mind that `allowDiskUse` will store the data into the disk rather than the memory, which might result in slower performance.
- The `db.aggregate()` command can return either a cursor or store the results in a collection. When returning a cursor or storing the results in a collection, each document in the result set is subject to the BSON Document Size limit, currently 16 megabytes; if any single document exceeds the BSON Document Size limit, the command will produce an error.
- If you have multiple stages in your pipeline, it's always better to understand the overhead associated with each stage. For instance, if you have both `$sort` and `$match` stage in your pipeline, it's highly recommended that you use a `$match` before `$sort` in order to minimize the documents that need to be sorted.

So, how can we optimize MongoDB for a faster aggregation pipeline? It depends on how the data is stored in the embedded document. If you store a million entries into the embedded document, it will create a performance overhead in stages like `$unwind`. (For faster querying, you might want to look into the [multi-key indexing](#) and [sharding](#) approaches for MongoDB and [especially understand how to optimize a pipeline](#).)

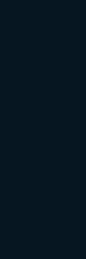
Ready to get started?

Launch a new cluster or migrate to MongoDB Atlas with zero downtime.

[Try Free](#)

Related Resources

- [Beginners guide: MongoDB basics](#)
- [Create a database in MongoDB](#)
- [What are clusters in MongoDB](#)
- [Scaling MongoDB](#)
- [Run MongoDB on AWS](#)
- [How to run MongoDB on Azure](#)



MongoDB.

About

Careers

Investor Relations

Legal

GitHub

Security Information

Trust Center

Connect with Us

Support

Contact Us

Customer Portal

Atlas Status

Customer Support

Manage Cookies

Deployment Options

MongoDB Atlas

Enterprise Advanced

Community Edition

Contact Us